Examen de prácticas PCD — Simulacro 2 (Puerto deportivo)

Nov 4, 2026 · @Laura

Prácticas de Programación Concurrente y Distribuida · 3º Curso de Grado en Ingeniería Informática · EXAMEN (simulacro)

Duración orientativa: 1 h 45 min. Este simulacro usa lo visto en las prácticas 1 a 6: hilos con Thread y Runnable, `synchronized`, `wait()` / `notify()` / `notifyAll()`, semáforos con `java.util.concurrent.Semaphore`, y Frame con Canvas.

Consideraciones previas

- No se permite el uso de ningún tipo de documentación.
- El acceso a Internet está desactivado conscientemente.
- Apague el teléfono móvil.

Antes de comenzar el examen

- Cree una carpeta con su nombre y primer apellido en el Escritorio separados por un guion bajo (ejemplo: Laura_Mata).
- En dicha carpeta aparecerá un proyecto por cada una de las preguntas del examen. Dichos proyectos se denominarán Proyecto1, Proyecto2, Proyecto3 y Proyecto4.

Enunciado

Un puerto deportivo dispone de un pantalán con 6 amarres. Al puerto llegan veleros y yates. Un velero ocupa 1 amarre, pero un yate, por su tamaño, ocupa 2 amarres a la vez. Por el ancho de la bocana del puerto, como máximo puede haber 4 veleros y 2 yates amarrados simultáneamente, y nunca se pueden superar los 6 amarres.

Los veleros tienen prioridad: mientras haya veleros esperando y todavía quepa alguno más de velero (hay menos de 4 dentro), los yates no podrán amarrar, aunque haya amarres libres.

Proyecto 1

Tiempo estimado: 40 minutos. Puntos: 4.

Será el proyecto base para solucionar el enunciado. Contendrá las siguientes clases:

- **Puerto.** Mantendrá el estado de ocupación del pantalán e implementará los siguientes métodos:
  - **EntraVelero.** Deberá ser invocado por los veleros cuando quieren amarrar.
  - **SaleVelero.** Deberá ser invocado por los veleros al abandonar el puerto.
  - **EntraYate.** Deberá ser invocado por los yates cuando quieren amarrar.
  - **SaleYate.** Deberá ser invocado por los yates al abandonar el puerto.
- **Velero.** Representará a cada velero mediante un hilo. El hilo se creará heredando de la clase Thread. Pondrá un mensaje de inicio indicando su identificador, intentará amarrar usando la clase Puerto, permanecerá amarrado un tiempo aleatorio entre 2 y 4 segundos y pondrá un mensaje al marcharse.
- **Yate.** Representará a cada yate mediante un hilo. El hilo se creará implementando el interface Runnable. Pondrá un mensaje de inicio indicando su identificador, intentará amarrar usando la clase Puerto, permanecerá amarrado un tiempo aleatorio entre 3 y 5 segundos y pondrá un mensaje al marcharse.
- **Generador.** Contendrá el método main y será quien comience la ejecución. Debe lanzar 20 barcos a intervalos de tiempo de entre 1 y 2 segundos. El tipo de barco será aleatorio, con una probabilidad del 60% para veleros y del 40% para yates. La ejecución finalizará cuando todos los hilos hayan finalizado.

Cada vez que un barco amarre o se marche, se mostrará un mensaje con su identificador y el número de amarres libres en ese momento.

El control de la concurrencia y la sincronización se realizará en la clase Puerto, mediante las primitivas de Java `wait()`, `notify()` y/o `notifyAll()`.

Proyecto 2

Tiempo estimado: 25 minutos. Puntos: 3.

Tomará como base el Proyecto 1. Con la prioridad de los veleros, un yate puede quedarse esperando indefinidamente si no dejan de llegar veleros. Para evitarlo, se modificará la clase Puerto de la siguiente forma:

- Cuando hayan amarrado 3 veleros seguidos mientras había algún yate esperando, los veleros perderán la prioridad hasta que amarre un yate.
- En cuanto amarre un yate, los veleros recuperarán la prioridad y la cuenta volverá a empezar.

Cuando un yate amarre gracias a esta regla, se mostrará un mensaje que lo indique.

El control de la concurrencia y la sincronización se seguirá realizando en la clase Puerto, mediante `wait()`, `notify()` y/o `notifyAll()`.

Proyecto 3

Tiempo estimado: 20 minutos. Puntos: 2.

Tomará como base el Proyecto 1 y se eliminará la clase Puerto. El control de la concurrencia se realizará únicamente mediante semáforos de la clase `java.util.concurrent.Semaphore`, que creará Generador y pasará a los barcos.

En este proyecto no habrá ninguna prioridad, pero se deberán seguir cumpliendo todos los límites: 6 amarres, 4 veleros y 2 yates como máximo. Un yate deberá ocupar sus 2 amarres de una sola vez, nunca primero uno y después el otro.

No se permite crear ninguna otra clase aparte de Velero, Yate y Generador.

Proyecto 4

Tiempo estimado: depende de la implementación que se pretenda. Puntos: 1.

Se creará un Frame que visualice de forma gráfica, mediante un Canvas, la situación del puerto y las colas de espera del Proyecto 1.

El nivel de representación gráfica es libre, pero al menos se deberán ver:

- La cola de veleros y la cola de yates, con el identificador de cada barco.
- Los 6 amarres, indicando en cada uno el identificador del barco que lo ocupa y de qué tipo es. Un yate ocupará 2 amarres en el dibujo.
- El número de veleros y de yates amarrados en cada momento, junto a su máximo (por ejemplo, "Veleros 3/4").